

1. Архитектура

Компонент	Технология	Роль
Nginx	nginx:1.27-alpine	Точка входа на порту 8084, reverse-проxy к Tracker
Tracker	Python (стандартная библиотека)	Фиксирует переход в SQLite и отдаёт учебную страницу
Mailer	Python (стандартная библиотека)	Читает CSV, генерирует токены, рассылает письма по SMTP
Mailpit	axllent/mailpit	Локальный SMTP-сервер (Mail Service) + web-интерфейс писем
SQLite	файл data/phishing.db	Хранилище зафиксированных переходов

Поток запроса при переходе по ссылке:

Пользователь → Nginx:8084 → Tracker:5000 → запись в SQLite → учебная страница

Ключевое техническое решение. Nginx сделан единой точкой входа и проксирует запрос в Tracker, а не отдаёт статический HTML напрямую. Это позволяет в рамках одного пользовательского запроса и зафиксировать переход в БД, и показать страницу. Реальный IP клиента Nginx передаёт в Tracker через заголовок X-Real-IP / X-Forwarded-For.

2. Инструкция по запуску

2.1. Запуск инфраструктуры (Задача 1)

```
docker compose up -d
```

Поднимаются сервисы tracker, nginx, mailpit.

Сервис	Адрес
Учебная страница	http://localhost:8084
Web-интерфейс почты (Mailpit)	http://localhost:8025

2.2. Рассылка учебных писем (Задача 4)

Список получателей — в mailer/users.csv. Запуск одноразового сервиса:

```
docker compose run --rm mailer
```

Для каждого пользователя генерируется уникальный token и персональная ссылка вида `http://localhost:8084/?email=david@test.ru&token=8f34d2....`. Отправленные письма видны в Mailpit (`http://localhost:8025`).

2.3. Переход по ссылке и учебная страница (Задачи 2–3)

Открыть письмо в Mailpit → перейти по ссылке. Появится страница-уведомление, а переход зафиксировается в БД.

2.4. Просмотр зафиксированных переходов

```
docker compose exec tracker python -c \
    "import sqlite3; [print(r) for r in
sqlite3.connect('/data/phishing.db').execute('SELECT email, token, ip,
created_at FROM clicks')]"
```

2.5. Остановка

```
docker compose down
```

3. Соответствие требованиям и подтверждение работы

Ниже — фактические выводы команд, снятые с работающего стенда.

3.1. Все сервисы запущены (Задача 1)

NAME	IMAGE	SERVICE	STATUS
phishing-campaign-mailpit-1 (healthy) 8025->8025	axllent/mailpit:latest	mailpit	Up
phishing-campaign-nginx-1 8084->8084	nginx:1.27-alpine	nginx	Up
phishing-campaign-tracker-1 5000/tcp	phishing-campaign...	tracker	Up

Комментарий: все четыре компонента (Tracker, Nginx, Mail Service, SQLite) стартуют без ошибок одной командой.

3.2. Учебная страница на порту 8084 (Задача 2)

Запрос `curl http://localhost:8084/?email=user@test.ru&token=123456`
возвращает страницу с текстом:

Вы перешли по учебной фишинговой ссылке. Если данное письмо вызвало подозрение — сообщите об этом в отдел информационной безопасности.

[Скриншот 1: учебная страница в браузере — вставьте свой снимок]

3.3. Фиксация переходов в SQLite (Задача 3)

После перехода по `http://localhost:8084/?email=ivan@test.ru&token=...` в БД появляется запись:

```
('david@test.ru', '8f34d2', '172.18.0.1', '2026-09-16 16:19:50')
('ivan@test.ru', '51d8044594b914d8', '172.18.0.1', '2026-09-16
16:21:36')
```

Сохраняются все требуемые поля: email, token, IP, дата и время.

[Скриншот 2: вывод SELECT из базы данных — вставьте свой снимок]

3.4. Сервис рассылки (Задача 4)

Вывод docker compose run --rm mailer:

```
[*] Отправка писем: 3 получател(ь/я/ей)
[*] SMTP: mailpit:1025
    -> david@test.ru    token=a294a8d2edd3ea5c
        http://localhost:8084/?email=david
%40test.ru&token=a294a8d2edd3ea5c
    -> ivan@test.ru     token=51d8044594b914d8
        http://localhost:8084/?email=ivan
%40test.ru&token=51d8044594b914d8
    -> maria@test.ru    token=d0c498828229c069
        http://localhost:8084/?email=maria
%40test.ru&token=d0c498828229c069
[+] Готово.
```

Подтверждается: чтение CSV, генерация уникальных токенов, формирование персональных ссылок, отправка писем через SMTP.

[Скриншот 3: список писем в Mailpit (http://localhost:8025) — вставьте свой снимок]

4. Структура проекта (Задача 5)

```
phishing-campaign/
├── docker-compose.yml
├── .gitignore
├── tracker/
│   ├── app.py
│   ├── test_tracker.py
│   └── Dockerfile
├── mailer/
│   ├── send.py
│   ├── users.csv
│   └── Dockerfile
├── nginx/
│   └── nginx.conf
└── data/
```

5. Принятые технические решения

- Mailpit как Mail Service. Локальный SMTP-сервер с web-интерфейсом не требует реальных учётных данных и секретов, что делает стенд полностью self-contained и безопасным для учебной среды. При необходимости отправку можно переключить на реальный SMTP (Gmail и т.п.) через переменные окружения SMTP_HOST/PORT/USER/PASS — при наличии SMTP_USER включается STARTTLS.

- Nginx как reverse-proxy. Обеспечивает единую точку входа на порту 8084 и передачу реального IP клиента, позволяя фиксировать переход и показывать страницу в одном запросе.
- Tracker на стандартной библиотеке (`http.server` + `sqlite3`) — одному GET-эндпоинту не нужны ни веб-фреймворк, ни внешние зависимости, ни `pip`-слой в образе.
- `secrets.token_hex` для генерации токенов — криптографически стойкая уникальность ссылок.
- Одноразовый сервис `mailer` через `profiles` — не стартует вместе со стендом, запускается явно командой рассылки.